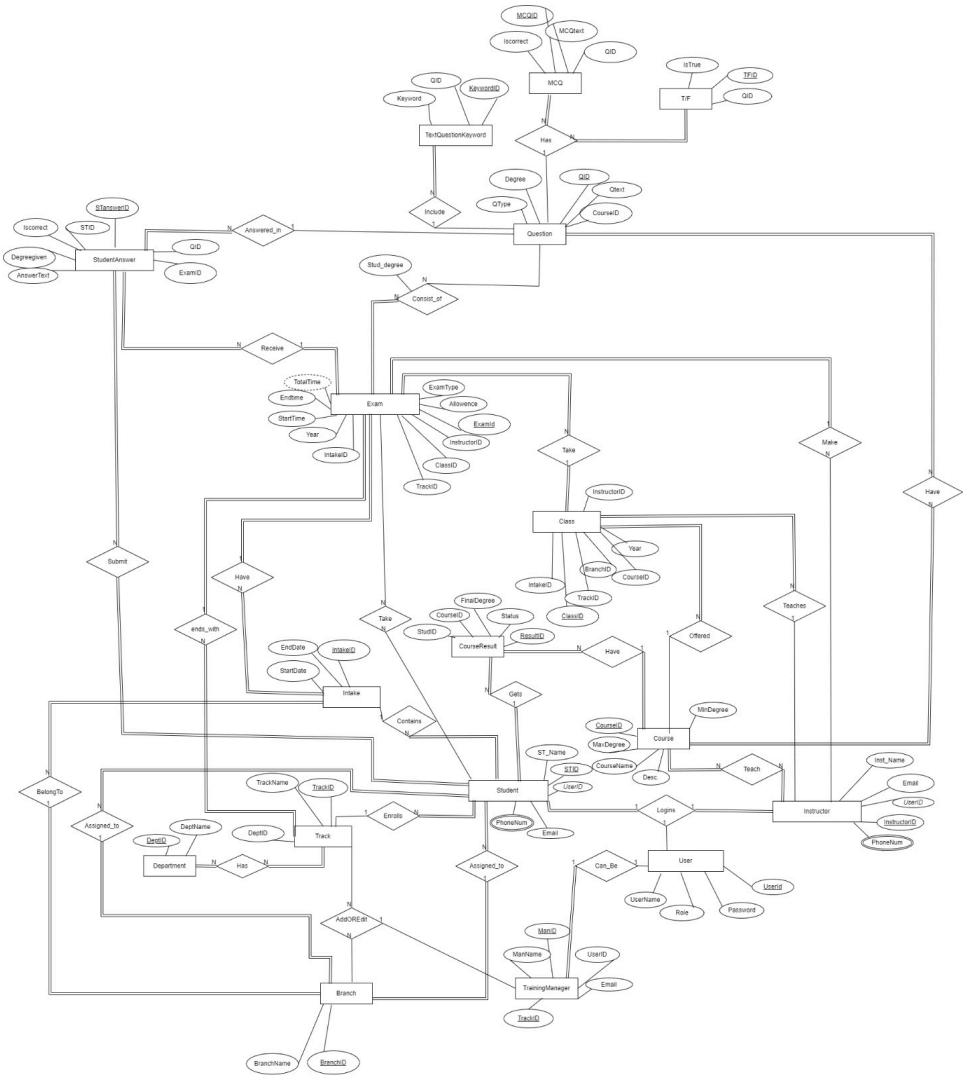
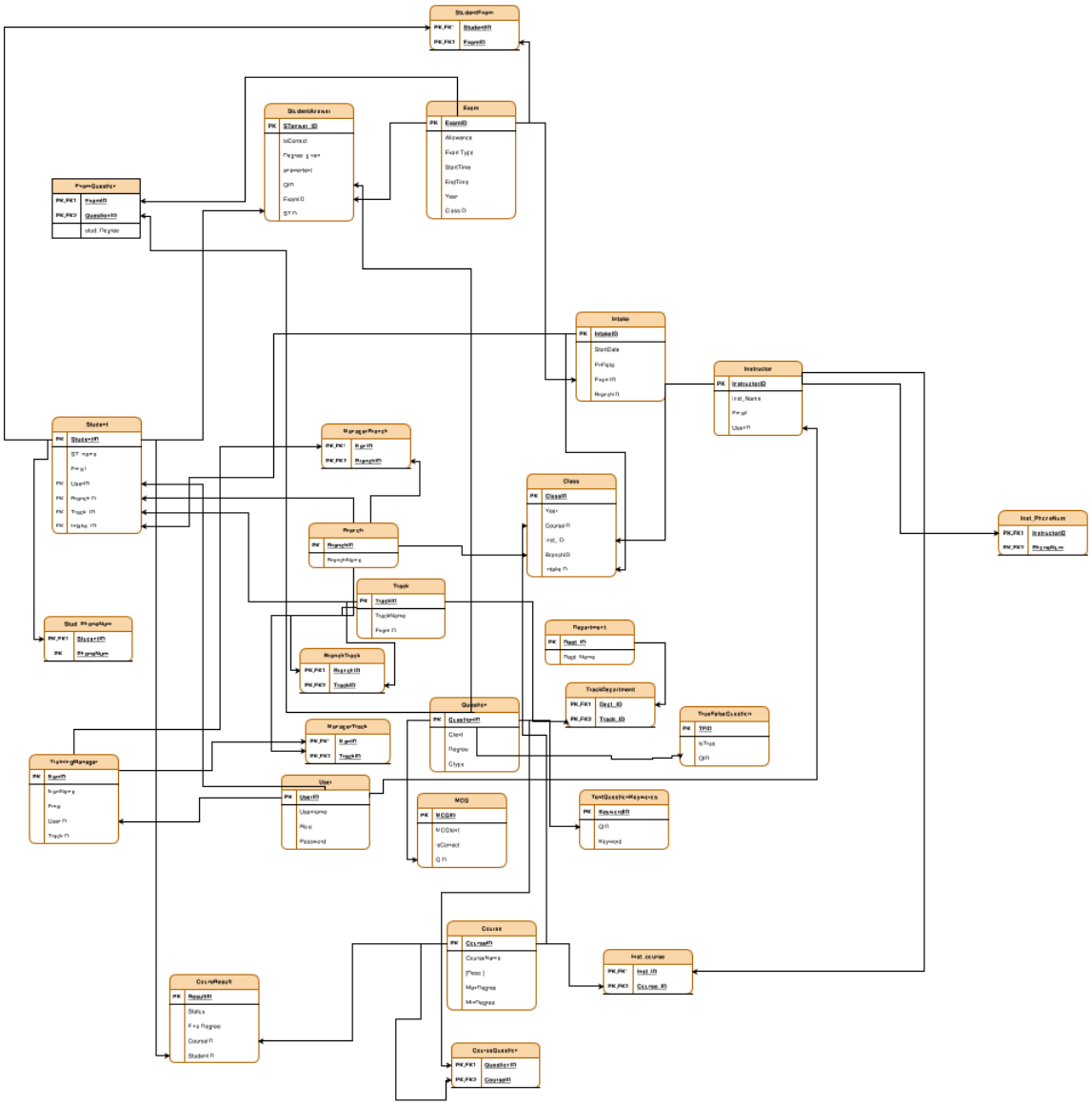
Examination System Roadmap

-رودماب للمشروع بإذن الله:

▼ 1 ERD



▼ 1.1 Mapping



▼ 1.2 Creating Filegroups

Filegroup Name	Purpose	Tables to Include
`USERDATA_FG`	Core entities (People/Org)	`Student`, `Instructor`, `User`, `TrainingManager`, `Branch`, `Department`, `Track`, `Intake`, `Class`
`CONTENT_FG`	Educational Content	`Course`, `Question`, `MCQ`, `TrueFalseQuestion`, `Exam`, `ExamQuestion`, `CourseQuestion`
`TRANSACTION_FG`	High-Frequency Logs	`StudentAnswer`, `StudentExam`, `CourseResult`, `Inst_PhoneNum`, `Stud_PhoneNum`

▼ 1.3 Creating tables (with columns, data types, primary keys, foreign keys)

In this step we create tables structure, columns with their datatypes, primary keys, foreign keys references, and basic constraints.

▼ 2 Constraints

Table Name	Target Columns	Constraint Logic & Reason
Course	`MinDegree`, `MaxDegree`	`MinDegree <= MaxDegree` Prevents logical errors where the passing score is higher than the total score.
Course	`MinDegree`, `MaxDegree`	`Values >= 0` Degrees cannot be negative.
Exam	`StartTime`, `EndTime`	`EndTime > StartTime` An exam cannot finish before it starts.
Exam	`ExamType`	`IN ('Normal', 'Corrective')` Restricts input to the types defined in requirements.
Question	`Qtype`	`IN ('MCQ', 'TF', 'Text')` Restricts questions to the three specific types allowed.
Question	`Degree_given`	`Degree > 0` Every question must carry a positive mark value.
User	`Role`	`IN ('Admin', 'Manager', 'Instructor', 'Student')` Limits access to the four defined system roles.
User	`Username`	`UNIQUE` Ensures no two users share the same login credential.
CourseResult	`FinalDegree`	`FinalDegree >= 0` Student scores cannot be negative.

StudentAnswer	`STID`, `ExamID`, `QID`	`UNIQUE` Ensures one answer per question.

▼ 2.1 Triggers:

1. Trigger to: ensure given question-grade inserted/updated is not bigger than the max degree of this question and also is not negative ✅
2. Trigger to ensure: inserted answer by student is between start and end time of the exam ✅
3. Trigger to ensure: no edit of any exam question between start and end time (Active Exam) ✅
4. Trigger to ensure: total grade for an exam can't be greater than the max course degree ✅
5. Trigger to check when a student submits an answer to an exam that he is registered to this exam ✅
6. Trigger to check we can't have same instructor for the same class (Same Branch, Intake, Year, Course) ✅
7. Trigger to check the exam time is within the intake interval ✅
8. Trigger to ensure no overlapping exams in the same intake ✅

▼ 3 Non_Clustered Indexes:

We need to look at columns we may need in joins, and filtering to add Non_Clustered Indexes on them.

In SQL Server, defining a Foreign Key does **not** automatically create an index on it. Without an index, any `JOIN` operation or check (like `ON DELETE CASCADE`) involving these columns will force the server to scan the entire table, which kills performance.

1. The Hidden Performance Killers (Composite Keys)

These tables have a Primary Key made of two columns (e.g., `Col A, Col B`). The Primary Key (Clustered Index) automatically created for `Col A`. However, `Col B` **is left** and needs its own Non-Clustered Index to support searches coming from that direction.

Table	Composite PK	Column Needing Non-Clustered Index
CourseHaveQs	`QuestionID, CourseID`	**`CourseID`**
TrackDepartment	`Dept_ID, Track_ID`	**`Track_ID`**
ExamQuestion	`ExamID, QuestionID`	`QuestionID`
Inst_course	`Inst_ID, Course_ID`	**`Course_ID`**
ManagerBranch	`ManagerID, BranchID`	**`BranchID`**
ManagerTrack	`ManagerID, TrackID`	**`TrackID`**
StudentExam	`StudentID, ExamID`	**`ExamID`**

2. The Transactional Foreign Keys (High Priority, heavy transactions)

These tables likely contain the most data (Students taking exams, answering questions). Missing indexes here will cause the biggest slowdowns in reporting and grading.

Table: StudentAnswer → (PK: answer_ID)

- `ExamID`
- `QuestionID`
- `StudentID`

Table: CourseResult → (PK: ResultID)

- `CourseID`
- `StudentID`

3. The Structural Foreign Keys (Standard Joins)

These columns are used to link entities (like finding which department a class belongs to).

Class (ClassID): `BranchID`, `CourseID`, `Inst_ID`, `IntakeID`

Student (STID): `BranchID`, `UserID`

Instructor (InstructorID): `UserID`

TrainingManager (ManID): `UserID`

Intake (IntakeID): `ExamID`

MCQ (MCQID): `QuestionID`

TrueFalseQuestion (TFID): `QuestionID`

▼ 4 Adding Sample Test Data

In this step we add basic sample data into tables to test constraints, and triggers.

▼ 5 Stored Procedures:

1. Training Manager SPs

Add/Edit a branch ✅

Remove a branch ✅

Add/Edit a track ✅

Remove a Track ✅

Add/Edit a intake ✅

Remove a Intake ✅

Add/Edit a student ✅

Remove a student ✅

Add/Edit/Remove an instructor (if not assigned with any classes) ✅

Add/Edit/Remove a course (if not assigned with any classes) ✅

Add/Edit/Remove a class (with hard delete option) ✅

2. Instructor SPs

Manage question pool: Insert a new Question ✅

Edit a question or an answer ✅

Remove a question from the question pool if not assigned with any exam ✅

Create an Exams ✅

Remove an Exams (if date is not met yet) ✅

View an Exam at any time ✅

Remove a student or (all-track-students) from an exam (if date is not met yet) ✅

Assign a students to an exam ✅

Grade_exam (with a solid logic for grading answers according to their types) ✅

3. Student SPs

View an exam (in the allowed time, should be registered for this exam) ✅

Submit an answers (with a solid logic for processing and saving answers according to their types) ✅

▼ Exam_Grade SP logic:

1. Grading each question according to the question type.
2. Calculate the Total Score by summing up the degrees of all correct answers for the exam.
3. Update the `CourseResult` table with the final score and pass/fail status.
4. For text questions get keywords first: For a question like *"Explain SQL"*, we get its keywords: `Structured`, `Query`, `Language` from the `TextQuestionKeywords` table,
5. Pattern Match: We check if the Student's Answer contains these words using `LIKE '%Keyword%'`.
6. Calculate Score:
   - If the student mentions 3 out of 3 keywords → 100% Grade.
   - If 2 out of 3 → 66% Grade, etc....

▼ 6 Views:

Admin Views:

- v_adm_AllInstructor
- v_adm_AllManager
- v_adm_AllStudent

- v_adm_AllUsers
- v_adm_DatabaseUsageStatistics
- v_adm_SystemActivitySummary

Training Manager Views:

- v_man_Branches
- v_man_IntakeDistribution
- v_man_Intakes
- vw_CourseClasses
- v_man_StudentPerformanceRanking
- v_man_StudentsPerTrackBranch
- v_man_TrackPerformanceSummary
- v_man_TracksPerDepartment

Instructor Views:

- v_Ins_InstructorCourses
- v_ins_CourseDetails
- v_Ins_InstructorExamList
- v_Ins_QuestionBankByInstructor
- v_ins_ExamDetails
- View_ExamQuestions
- View_QuestionsWithModelAnswers
- View_StudentGrades

Student Views:

- v_stu_AvailableExams
- v_stu_StudentAnswerReview
- v_Stu_StudentExamResults

▼ 7. Logins → Users → Roles permissions container → Add users to Roles

- **Login:** This handles authentication at the server instance level.
- **User:** We use it to manage access on databases, and defining the default schema for each user.
- **Roles:** These act as containers for permissions.
  - **Server Roles:** These operate at the server level, such as `sysadmin`.
  - **Database Roles:** Such as `insert, update, delete`.
- We first create the role, assign the appropriate permissions to it, and then alter the role to add the member (users).

▼ 8 (Adding Backup Job)

How this works:

- **Dynamic File Naming:** Generates a little script *inside* the job that adds the current date to your backup file (like `ExamsSystem_20260219.bak`). This prevents it from overwriting yesterday's backup.
- `@freq_type = 4`: Tells SQL Server this is a daily recurring schedule.
- `@active_start_time = 060000`: This time without colons (06 hours, 00 minutes, 00 seconds).

▼ 9. Test Cases

Test cases for Triggers, SPs, Views, and Roles.

تم بحمد الله وتوفيقه.